Sanitized Public Review Copy

This text-edition PDF was generated for publication review. Student IDs were removed and the public author name was normalized to Yasir Alharbi. Original source files were not modified.

Page 1

CS-616 SOC Alert Triage Optimization

Page 1

Data-Driven SOC Alert Triage Scheduling in a Shift

MILP Optimization with Gurobi, ML Predictions, and Simulated Annealing

Field Details

Author Yasir Alharbi

Student ID [student ID removed]

Course CS-616 Optimization Algorithms

Institution Prince Sattam bin Abdulaziz University

Project Focus SOC alert triage scheduling under real shift constraints

Abstract

Security Operations Centers receive more alerts than analysts can investigate with equal attention. This project models alert triage as a constrained scheduling problem: choose which alerts should be handled, which analyst should handle them, and when they should be handled during one 8-hour shift. The final notebook uses machine learning to estimate alert risk, required skill, and investigation duration, then sends those predictions into an optimization model. The MILP formulation is solved with Gurobi, while Simulated Annealing is used as a metaheuristic comparison. The updated model uses 60 minutes of workload capacity per analyst-hour, so one analyst can handle multiple short alerts when the total predicted time fits. It also treats L3 as an on-call escalation resource for confirmed incidents rather than normal shift capacity. In the final 300-second Gurobi experiment, large instances reached the time limit but had optimality gaps below 0.2%, which indicates near-optimal solutions for practical SOC planning.

1. Introduction

The project started from a simple SOC reality: alerts do not wait politely for analysts to be free. Some alerts are noisy, some are urgent, and some require skills that not every analyst has. A useful triage system therefore needs more than a priority list. It needs a feasible plan that

respects shift capacity, breaks, skills, deadlines, and escalation policy.

In this work, I treated SOC alert triage as an optimization problem. The decision is not only whether an alert looks risky. The decision is whether the SOC can assign that alert to the right analyst at the right time without breaking operational rules. This makes the project a prediction-to-decision pipeline: machine learning estimates important alert properties, and optimization turns those estimates into a schedule.

2. Current Project Pipeline

Stage Role in the project

Synthetic alert generation Creates realistic alert features such as severity, deadline, skill need, asset criticality,

Page 2

false-positive history, alert type, incident label, and investigation duration.

ML risk scoring Uses Logistic Regression to estimate the probability that an alert is a true incident.

This becomes the objective weight.

ML skill prediction Uses a multiclass Logistic Regression model to predict whether the alert needs

general triage, DFIR, PenTest, or MalwareRE capability.

ML duration prediction Uses Ridge Regression to estimate investigation minutes. This drives the 60-minute

workload capacity constraint.

MILP optimization Uses Gurobi to choose alert-to-analyst-to-slot assignments while enforcing deadlines,

breaks, skills, capacity, and L3 policy.

Simulated Annealing Provides a fast heuristic comparison using the same feasibility logic and repeated

stochastic runs.

Important modeling update: The current project no longer assumes one alert per analyst-hour. It uses

predicted investigation minutes, so an analyst may handle several short alerts in the same hour as long as

their total predicted workload is at most 60 minutes.

3. Problem Definition

The project models one 8-hour SOC shift divided into eight one-hour slots. There are three

active on-shift analysts: two L1 analysts and one L2 analyst. L3 is included only as an on-call

escalation resource. This distinction matters because L3 should not be counted as normal shift

capacity and should not receive ordinary queue alerts.

The model must satisfy these operating rules:

• Each alert may be assigned at most once; if capacity or skills are not enough, some alerts

remain queued.

• For each on-shift analyst and hour, total assigned predicted investigation minutes must be at

most 60.

• Alerts must be assigned no later than their deadline slot.

• Analysts cannot receive alerts during break slots.

• Specialized alerts require compatible skills such as DFIR, PenTest, or MalwareRE.

• L2 can handle basic alerts when L1 analysts are busy, but specialized needs still matter.

• L3 is used only for confirmed incident escalation and is limited to rare on-call use.

4. Data and SOC Assumptions

Real SOC data is sensitive, so the notebook generates synthetic alerts with realistic structure. Each alert has a severity level, an SLA-derived deadline, a possible required skill, a critical-confirmed flag, an asset criticality score, false-positive history, alert source type, an incident label, and a true investigation duration used for training the duration model.

4.1 SLA Assumption

Severity Example alert SOC response time

Critical Active ransomware, confirmed data exfiltration, domain Immediate response, within 1

Page 3

admin compromise hour

High Suspicious PowerShell, credential spraying, impossible

travel login Within 2-4 hours

Medium Endpoint quarantine, unusual cloud token activity, repeated

failed logins Within 4-8 hours

Low Low-confidence SIEM rule, noisy alert, minor policy

violation Within 24 hours

In the notebook, this SLA logic is converted into a deadline slot. The optimizer can only place an alert in a slot that is before or equal to its deadline.

4.2 Analyst Skills and Capacity

Resource Level DFIR PenTest MalwareRE Role

L1_A 1 0 0 0 On-shift analyst

L1_B 1 0 0 0 On-shift analyst

L2_C 2 1 1 0 On-shift analyst

L3_OnCall 3 1 1 1 On-call escalation

The fixed-seed break schedule in the updated notebook is L1_A at slot 6, L1_B at slot 3, and L2_C at slot 5. These breaks apply only to the three normal on-shift analysts. L3 is separate and is not part of the normal shift capacity calculation.

5. Machine Learning Integration

The ML part is a bonus in the project, but it is also useful operationally. It does not replace optimization. Instead, it creates better input parameters for the optimizer. The final notebook uses three models:

Prediction Model Output column How optimization uses it

Incident risk Logistic Regression score Objective weight. Higher scores make

alerts more valuable to schedule.

Required skill Multiclass Logistic

Regression pred_req_skill

Skill compatibility constraint. The assigned

analyst must have the predicted required

skill.

Investigation

duration Ridge Regression pred_duration_min

Workload capacity constraint. Total

assigned minutes per analyst-hour must

be at most 60.

Categorical features are encoded with one-hot encoding, and numeric features are passed

through the preprocessing pipeline. This keeps the code clean because the same pipeline

handles mixed alert features such as severity, alert type, asset criticality, false-positive history,

and deadline.

Alerts Risk AUC Risk accuracy Skill accuracy Duration MAE Duration R2

50 0.733 0.692 0.385 4.21 min 0.707

Page 4

100 0.743 0.760 0.520 4.89 min 0.611

200 0.715 0.740 0.420 4.65 min 0.753

500 0.807 0.720 0.472 3.33 min 0.791

The duration model is especially useful for the final optimization model. A duration error of roughly 3 to 5 minutes is acceptable for this planning-level experiment because the capacity unit is an hour. Skill accuracy is lower than risk and duration performance, which is realistic because required skill can be ambiguous from alert metadata alone.

6. Mathematical Formulation

Let I be the set of alerts, A be the set of analyst resources, and T be the set of hourly time slots. The binary decision variable is x[i,a,t]. It equals 1 if alert i is assigned to analyst a in slot t, and 0 otherwise.

The objective maximizes the total risk score of assigned alerts. In words, the optimizer tries to use available capacity on the alerts that the ML model estimates as most important. An alert that remains queued contributes zero to the objective because its decision variable stays 0.

Constraint Meaning in the SOC model

Alert uniqueness Each alert is assigned at most once, not exactly once. This allows the queue to remain if the shift has insufficient feasible capacity.

Minute capacity For each analyst and hour, the sum of predicted investigation minutes cannot exceed 60.

Deadline An alert cannot be scheduled after its SLA-derived deadline slot.

Availability If an analyst is on break or unavailable, the assignment is not allowed.

Skill compatibility If an alert needs DFIR, PenTest, or MalwareRE, the assigned resource must have that capability.

L3 policy L3_OnCall is allowed only for confirmed incidents and is not treated as normal queue capacity.

Why at most once? The model uses <= 1 instead of = 1 because forcing every alert to be assigned could make the model infeasible or unrealistic. In a real SOC, some alerts may stay queued for the next shift when capacity, deadlines, or skills are not enough.

7. Exact Solution with Gurobi

The Gurobi implementation translates the formulation into code. Lists and dataframe columns represent sets and parameters, and Gurobi binary variables represent x[i,a,t]. Impossible assignments are filtered before variables are created. For example, the code does not create a variable if the alert would miss its deadline, if the analyst is on break, if the skill is incompatible, or if L3 is being used for a normal non-confirmed alert.

For the final reported run, Gurobi was given a 300-second time limit per instance. This is a practical optimization setting. The unlimited full solve can spend a long time proving optimality for larger instances, even after it has already found a strong feasible schedule. Therefore, the

Page 5

paper reports Gurobi status and optimality gap instead of pretending every large instance was fully proven optimal.

8. Simulated Annealing

Simulated Annealing is the metaheuristic comparison. In the code, a candidate solution is represented as a schedule dictionary where each analyst-slot pair contains a list of assigned alerts. This representation matches the updated minute-capacity model because more than one alert can appear in the same analyst-hour as long as their total predicted duration fits within 60 minutes.

The algorithm starts from a greedy feasible schedule, then explores neighboring schedules using move, remove, replace-low, assign-unassigned, swap, and destroy-and-repair operations. A better neighbor is accepted directly. A worse neighbor can still be accepted with probability exp(delta / T), which helps the search escape local optima early. As temperature decreases, the algorithm becomes more selective.

SA was run 10 independent times for each alert size. The paper reports the mean, standard deviation, best, worst, and average runtime because SA is stochastic and can produce slightly different solutions across runs.

9. Experimental Study and Results

The experiment compares Gurobi and ML-guided SA on 50, 100, 200, and 500 alerts. The final results below are from the updated 300-second notebook copy.

Alert

s

Gurobi

status

Gurobi

gap

Gurobi

obj.

Gurobi

time

Assig

ned

SA

mean

SA

best

SA

time SA mean gap

50 OPTIMAL 0.000% 17.938 0.04s 46 17.476 17.583 0.27s 2.57%

100 TIME_LIMI

T 0.175% 33.027 300.00s 77 28.954 29.529 1.34s 12.33%

200 TIME_LIMI

T 0.166% 50.839 300.00s 99 47.136 48.027 6.39s 7.28%

500 TIME_LIMI

T 0.137% 63.441 300.01s 110 57.686 57.686 5.69s 9.07%

The 50-alert case was solved to proven optimality almost immediately. For 100, 200, and 500 alerts, Gurobi reached the 300-second limit, but the optimality gaps were below 0.2%. This means the best feasible schedules were extremely close to the mathematical upper bound. The correct interpretation is near-optimal, not failed.

SA was much faster than Gurobi in runtime, but its objective value was lower. This is the expected trade-off: SA can produce a quick feasible schedule, while Gurobi gives stronger solution quality and a bound-based measure of how close the solution is to optimal.

Page 6

Figure 1. Runtime comparison between Gurobi and SA.

Figure 2. Objective comparison between Gurobi and SA mean.

10. Discussion

The strongest result is that the final model behaves like a realistic SOC decision-support tool. It does not simply rank alerts. It builds a feasible assignment plan. Risk scoring decides what is valuable, duration prediction decides how much work fits, skill prediction decides who is capable, and the optimizer checks whether the schedule is actually allowed.

Page 7

The Gurobi results show that exact optimization is very useful when quality and guarantees matter. Even when the solver reached the time limit, the remaining gap was very small. The SA results show why metaheuristics are still valuable: they are faster and flexible, but they sacrifice some solution quality. In a SOC, this suggests two possible modes. Gurobi can be used for careful planning and validation, while SA can support quick replanning when the environment changes.

The L3 update is also important. Treating L3 as a normal shift analyst would make the schedule look stronger than the real operation. The final notebook keeps L3 as an on-call escalation option for confirmed incidents only, which better matches how SOC teams usually protect senior specialist capacity.

11. Limitations and Future Work

The main limitation is that the data is synthetic. This is appropriate for a course project because SOC data is sensitive, but real alert data may have patterns that are not fully captured by the generator. Another limitation is that the model plans a single shift at a time. A production SOC tool would need online replanning as new alerts arrive, analyst status changes, and incidents evolve.

Future work could use anonymized real SOC data, add handoff rules between shifts, include analyst fatigue, add dynamic alert arrivals, and improve the skill prediction model with richer text or rule context. The current structure is a good base because it separates prediction from optimization: better ML models can be plugged in without changing the core scheduling formulation.

12. Conclusion

This project built a data-driven optimization model for SOC alert triage. The final version combines three ML predictions with two optimization methods. Logistic Regression estimates alert risk, multiclass Logistic Regression estimates required skill, Ridge Regression estimates duration, Gurobi solves the MILP assignment model, and Simulated Annealing provides a fast heuristic comparison.

The updated model is more realistic than the earlier one-alert-per-slot version because it uses

predicted minutes and a 60-minute capacity limit. It also keeps L3 as an on-call escalation resource instead of counting L3 as normal shift capacity. The final experiment shows that Gurobi can produce near-optimal large-instance schedules with very small gaps under a 300-second limit, while SA offers faster approximate schedules. Together, the methods demonstrate a complete prediction-to-decision workflow for SOC triage.

Appendix A. Alignment with CS-616 Requirements

Requirement	How it is addressed
Real-world problem	SOC alert triage scheduling with deadlines, skills, breaks, workload capacity, and L3 escalation policy.

Page 8

Mathematical model MILP with binary assignment variables, risk objective, and linear operational constraints.

Exact solution Gurobi implementation with status, runtime, assignment count, and optimality gap.

Metaheuristic ML-guided Simulated Annealing with feasibility checks and multiple neighborhood moves.

Experimental study Four alert sizes, 10 SA runs per size, runtime comparison, quality comparison, and gap analysis.

Technology bonus Three ML models: risk scoring, required-skill prediction, and duration prediction.

Notebook deliverable Single notebook pipeline with data generation, ML, MILP, SA, plots, and discussion.

References

1. S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," Science, vol. 220, no. 4598, pp. 671-680, 1983.

2. Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, 2026.

3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

4. M. L. Pinedo, Scheduling: Theory, Algorithms, and Systems, 5th ed., Springer, 2016.

5. NIST, Incident Response Recommendations and Considerations for Cybersecurity Risk Management, SP 800-61 Rev. 3, 2025.